Reunión Mensual **PhytoLearning**

UAI | FACULTAD DE INGENIERÍA Y CIENCIAS

Geometría y LLMs para clases con pocos ejemplos

Cómo elegir datos generados por IA

Benjamín Schindler

Laboratorio Dr. Gonzalo A. Ruz

Master of Science in Data Science
Universidad Adolfo Ibáñez

Hoja de ruta

1. **Introducción** — desbalance, voces que se callan y por qué las clases raras suelen ser las que más importan.
2. **Problema** — qué pasa cuando hay pocos ejemplos y qué se hace hoy al respecto.
3. **Estado del arte** — dos caminos para inventar ejemplos (el clásico y la IA) y lo que le falta a ambos.
4. **Hipótesis y objetivos** — qué afirmamos y cómo se va a verificar.
5. **Metodología** — un «mapa de significados», filtros de calidad sobre ese mapa y pesos por calidad.
6. **Resultados** — 8.000+ experimentos: la idea funciona, y lo simple le gana a lo complejo.
7. **Discusión** — utilidad práctica, limitaciones honestas y trabajo futuro.

1.
Introducción

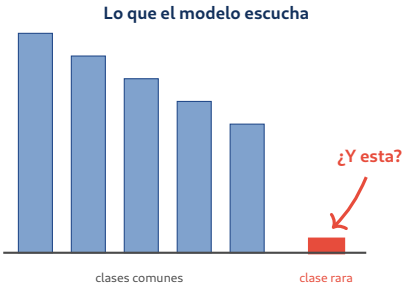
1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

¿Estamos escuchando todas las voces?

En la práctica, los datos **más importantes** suelen ser los **menos frecuentes**. La realidad rara vez se reparte por partes iguales:

- **Un** correo de phishing con un esquema nunca antes visto.
- Notas clínicas con síntomas raros: **un puñado** de menciones.
- **Un** mensaje de auxilio entre miles de consultas rutinarias.
- Discurso de odio en lenguas minorizadas: **decenas** de ejemplos.

Lo común tapa lo raro.
¿Y cómo afecta esto a los modelos de predicción?



¿Cómo aprende un modelo lo que apenas escucha?

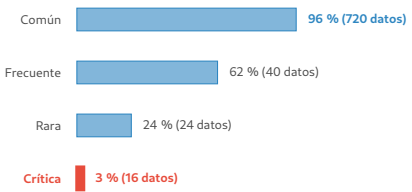
2.
Problema

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Por qué los clasificadores se inclinan hacia las clases mayoritarias

Un modelo puede ser **correcto en el 95 % de los casos** y, aun así, **fallar en el 100 % de los que importan**.

% de casos que el modelo detecta, por clase



Acierto global
95 %

Detección de la clase crítica
0 %

El mismo modelo.
Dos verdades.

La voz que el modelo **no escuchó**
suele cargar con el mayor **costo humano**.

¿Cómo le enseñamos a un modelo a reconocer aquello que casi no ha visto?

¿Qué se hace para este desbalance?

Desde hace décadas la respuesta es la misma: **emparejar la cancha** cambiando cuántos ejemplos ve el modelo de cada clase. Hay dos maneras.

Recortar lo común (undersampling)

Eliminar ejemplos de las clases grandes hasta equilibrar.

✓ Simple y rápido.
✗ Bota información — y si ya hay pocos datos, es un lujo imposible.

Crear más de lo raro (oversampling)

Inventar ejemplos nuevos para las clases chicas: duplicar, **SMOTE**, **LLMs**…

✓ Conserva todos los datos reales.
✗ Riesgo: ejemplos inventados de mala calidad.

Esta tesis vive en el segundo camino: y ahí hay dos formas de inventar — **mezclar lo que ya se tiene** o **crear texto nuevo con IA**.

¿Qué métodos existen, cómo funcionan y funcionan siempre?

3.
Estado del arte

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

La pieza clave: convertir cada texto en un punto de un mapa

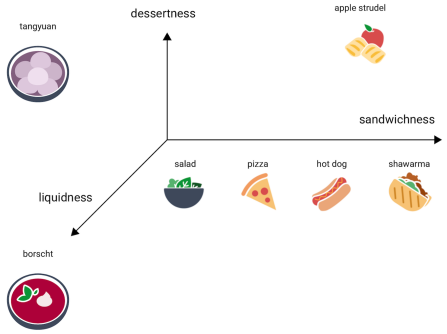
Los computadores no leen palabras: leen **números**.

Un modelo de **embeddings** le hace muchas preguntas a cada texto y lo convierte en **un punto de un mapa**.

La propiedad clave:

- Cosas con significado parecido quedan cerca.
- Las que no tienen nada que ver quedan lejos.

Todo lo que viene ocurre en ese mapa: cada oración, real o inventada, es **un punto**.



Con comidas bastan 3 preguntas. Con texto es igual, pero con **cientos**.

Imagen de Google, ML Crash Course (CC BY 4.0).

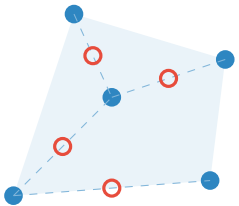
Con los textos convertidos en puntos, «parecerse» se vuelve algo que podemos medir.

Camino 1 — SMOTE: inventar puntos intermedios en el mapa

SMOTE [Chawla et al., 2002] es el método clásico (20+ años) para crear ejemplos de la clase escasa.

La idea, en una frase:

- Toma dos ejemplos reales que sean vecinos en el mapa…
- …y crea un punto nuevo en el camino entre ambos: un «promedio» de los dos.



Puntos nuevos entre vecinos

✓ Controlado: nunca se sale de la zona conocida.
✓ Simple y rápido; funciona hace décadas.

✗ Genera puntos, no texto: no hay oración nueva que leer.
✗ Solo rellena lo que ya se sabe: no aporta variedad nueva.

Control total sobre dónde caen los puntos… pero sin texto nuevo y sin nada realmente nuevo.

Camino 2 — Pedirle los ejemplos a una IA (LLM)

Pedirle a una IA generativa (ChatGPT, Gemini, Claude…) que **escriba ejemplos nuevos** para la clase que tiene pocos.

La receta que se usa hoy:

1. Mostrarle los pocos ejemplos reales que hay.
2. Pedirle: «escribe 50 más como estos».
3. Agregar **todo** lo que responda al entrenamiento.



Varios estudios reportan mejoras [Whitehouse 2023; Suhaeni 2024; Cegin 2025] — pero todos comparten el mismo punto ciego.

✓ Texto real, fluido y variado.
✗ Nadie revisa dónde caen esos textos en el mapa: se acepta todo, bueno y malo.

Texto rico pero sin control: los puntos caen donde quieran en el mapa.

El dilema: SMOTE vs. LLMs lado a lado

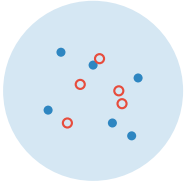
SMOTE / interpolación

✓ Sabemos **dónde** caen sus puntos

✓ Probado por más de 20 años

✗ No produce texto nuevo

✗ Solo rellena lo ya conocido



Todo queda dentro de la zona conocida

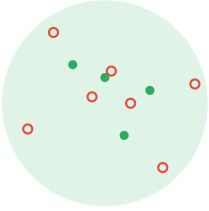
LLM (Gemini, GPT, Claude…)

✓ Texto fluido y variado

✓ Aporta contenido **nuevo**

✗ No sabemos dónde caen sus puntos

✗ Puede mezclar clases (ruido)



Los puntos caen donde sea

¿Podemos quedarnos con lo mejor de ambos mundos?

La idea de esta tesis: ponerle un control de calidad a la IA

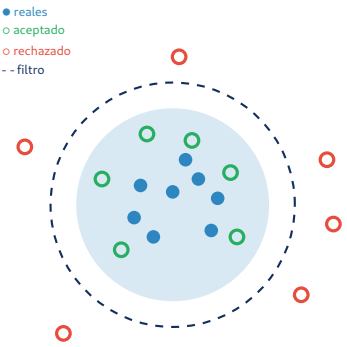
Nadie revisa dónde caen los textos que escribe la IA.
Eso abre una solución sorprendentemente simple.

La idea:

1. Pedirle a la IA **más ejemplos de los necesarios**.
2. Ubicar cada uno como un punto en el mapa.
3. Medir qué tan cerca cae de los **ejemplos reales** de su clase.
4. Descartar los que caen demasiado lejos.

¿Por qué tiene sentido?

- El mapa ya lo teníamos: es el mismo que usa el clasificador.
- Sirve cualquier IA (Gemini, GPT, Claude…): no hay que abrirla ni re-entrenarla.



Lo mejor de ambos mundos: el texto lo pone la IA, el control de calidad lo pone el mapa.

4.
Hipótesis y objetivos

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Hipótesis y objetivos

H1 — Hipótesis principal

Si a los ejemplos que escribe la IA les aplicamos un **control de calidad en el mapa** (quedarnos con los bien ubicados), el clasificador mejora **más que con SMOTE**, el método clásico — y la mejora es real, **no producto del azar**.

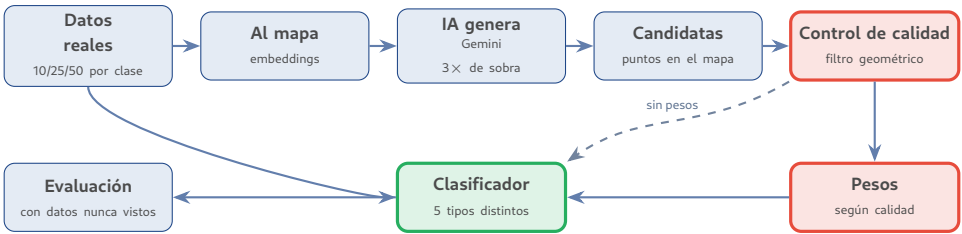
Objetivo general

Desarrollar y validar ese control de calidad de modo que funcione **mirando solo el mapa**: sin importar qué IA escribió el texto, y en escenarios con **pocos ejemplos por clase**.

5.
Metodología

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Pipeline: del texto real al clasificador entrenado



Las dos piezas nuevas (control de calidad y pesos) no tocan la IA: funcionan con cualquiera.

Protocolo experimental: 8.000+ experimentos

Probamos la idea en todas las combinaciones:

- **13 conjuntos de datos** en 9 dominios.
- **Escasez:** 10 / 25 / 50 ejemplos por clase.
- **5 tipos** de clasificador.
- **10 métodos** de aumentación en competencia.
- **5 IAs generadoras** (Gemini, GPT-5, Claude, Kimi, GLM).
- **4 mapas** (modelos de embeddings) distintos.
- **5 repeticiones** de cada experimento.

Cómo comparamos:

- Nota de calidad: **macro F1** (siguiente slide).
- Cada método compite contra **SMOTE** en el mismo escenario exacto.
- Pruebas estadísticas estrictas: descartan que la diferencia sea azar.
- También contamos **cuántas veces gana** cada método.

Volumen

8.000+ experimentos

¿Cómo medimos el éxito? Una nota justa: macro F1

El acierto global engaña: un modelo que responde siempre «clase común» acierta el 95 %… ignorando por completo a la clase rara.

Macro F1 hace algo más justo:

1. Le pone una **nota de 0 a 100 a cada clase**: qué tan bien la detecta, sin confundirla con otras.
2. **Promedia las notas**: cada clase pesa lo mismo, tenga 16 ejemplos o 720.

Como el promedio de notas por ramo: sacarse un 7 en el ramo fácil **no tapa** el 2 en el difícil.

Por qué nos sirve

- Un error en la clase rara **duele igual** que uno en la clase grande.
- Si el modelo ignora una clase chica, la nota cae de inmediato.

Cómo leerán los resultados

- ΔF1 = cuántos **puntos de nota** sube (o baja) un método frente a SMOTE.
- «pp» = puntos porcentuales: $+2$ pp = subir de 71 a 73.
- Siempre evaluado sobre datos que el modelo **nunca vio**.

Macro F1: la métrica honesta cuando hay clases raras.

Conjuntos de datos: 13 corpus en 9 dominios

Conjunto	Dominio	Clases	n-shots	Test
Clasificación de texto — experimento principal				
`sms_spam`	Detección de spam	2	10, 25, 50	1.394
`hate_speech_davidson`	Discurso de odio	3	10, 25, 50	4.948
`20newsgroups`	Noticias (4 cat.)	4	10, 25, 50	7.532
`ag_news`	Noticias (4 cat.)	4	10, 25, 50	7.600
`emotion`	Emociones	6	10, 25, 50	2.000
`dbpedia14`	Clasificación de temas	14	10, 25, 50	5.000
`20newsgroups_20class`	Noticias (20 cat.)	20	10, 25, 50	7.532
Adicionales — escalabilidad por número de clases				
`trec6`	Preguntas	6	10, 25, 50	500
`banking77`	Intenciones bancarias	77	10, 25, 50	3.080
`clinc150`	Intenciones de diálogo	150	10, 25, 50	4.500
Reconocimiento de entidades nombradas (NER)				
`MultiNERD`	Entidades generales	5 tipos	10, 25, 50	10.000
`WikiANN`	Entidades Wikipedia	3 tipos	10, 25, 50	10.000
`Few-NERD`	Entidades detalladas	8 tipos	10, 25, 50	10.000

n-shot estricto: 10 / 25 / 50 ejemplos por clase. Rango de complejidad: de binario (*`sms_spam`*) a 150 clases (*`clinc150`*).

¿Qué es NER y por qué la usamos?

NER (Named Entity Recognition)

Tarea de NLP que **detecta nombres propios dentro del texto y los clasifica por tipo**: personas, lugares, organizaciones, fechas, etc.

Ejemplo: «Llamé a María en Madrid el 2 de marzo .» ↑PER ↑LOC ↑DATE

¿Por qué la usamos como prueba?

- Construimos el filtro para **clasificación**.
- NER es una tarea NLP **distinta**: en vez de etiquetar la oración entera, se etiquetan palabras dentro.
- Si el mismo filtro funciona en NER **sin cambiarlo** ⇒ el filtro no depende de la tarea.

Es la prueba más fuerte de que el método **generaliza**.

3 corpus, distinta complejidad

- `WikiANN` — 3 tipos (PER, ORG, LOC). **Fácil**.
- `MultiNERD` — 5 tipos (+ DATE, MISC). Medio.
- `Few-NERD` — 8 tipos jerárquicos. **Difícil**.

Mismo filtro, otra tarea: si funciona acá, sirve en general.

Cuatro maneras de hacer el control de calidad

Probamos cuatro «porteros» distintos, del más permisivo al más estricto:

Filtro	La pregunta que le hace a cada candidata
Ninguno (control)	Deja pasar todo. Sirve de punto de comparación.
LOF (densidad)	¿Caíste en una zona **poblada** de tu clase, o en un descampado?
Cascada (niveles 1–4)	¿Qué tan **cerca** caíste de los ejemplos reales? Ordena a todas las candidatas y se queda con las mejores.
Combinado	Exige **dos condiciones a la vez** (densidad **y** parecido). El portero más estricto.

Mismo mapa, cuatro criterios: ¿cuál elige mejor?

La cascada: cuatro preguntas sobre cada candidata

Cada texto que escribe la IA recibe un **puntaje de calidad** según dónde cayó su punto en el mapa. Cuatro preguntas, de la más simple a la más elaborada:

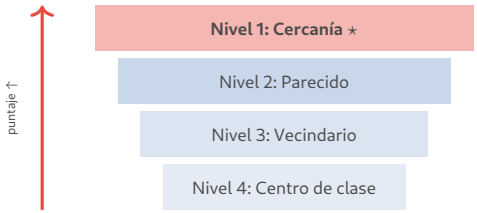
Nivel 1 ⋆ **Cercanía:** ¿qué tan cerca caíste del ejemplo real más próximo?

Nivel 2 **Parecido:** ¿te «pareces» a ese ejemplo? (similitud de significado)

Nivel 3 **Vecindario:** ¿tus vecinos en el mapa son de tu misma clase?

Nivel 4 **Centro:** ¿qué tan cerca estás del centro de tu clase?

Las preguntas activas se combinan en un solo puntaje, y nos quedamos con las N **mejores** candidatas del ranking.



Se queda con las N **mejores** del ranking

(no hay nota mínima de corte).

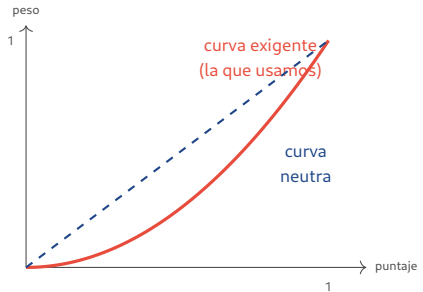
Adelanto: la pregunta más simple —el Nivel 1, pura cercanía— resultará ser la única necesaria.

Ponderación suave: no todas las muestras valen lo mismo

La segunda contribución: en vez de **aceptar o rechazar** en seco, cada muestra entra al entrenamiento con un **peso según su puntaje**.

- **Ejemplo real** ⇒ peso máximo (1): en ellos confiamos siempre.
- **Candidata bien ubicada** ⇒ peso alto.
- **Candidata dudosa** ⇒ peso bajo: se usa, pero se le cree menos.

Como un profesor que escucha a todo el curso, pero pondera más las respuestas de quienes muestran su desarrollo.



La curva exigente castiga fuerte a las dudosas y premia a las muy bien ubicadas.

El clasificador escucha a todas las muestras, pero les cree en proporción a su calidad.

6.
Resultados

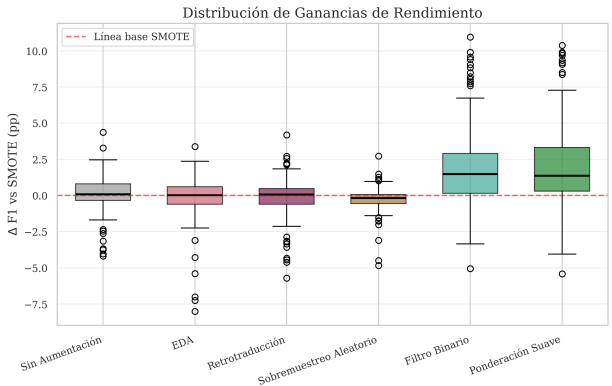
1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Resultado principal

+2,25 pp

puntos de macro F1 sobre SMOTE

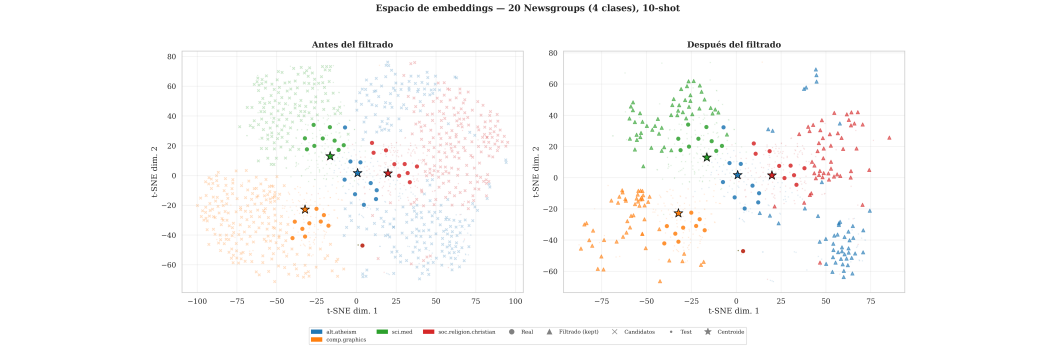
- En la nota: de **71.2** a **73.5**.
- Gana en el **83.8 %** de los casos.
- Real, no azar: $p < 0.0001$.
- Efecto de tamaño medio-grande.
- $N = 3.675$ experimentos.



Distribución de ΔF1 por método.

El método propuesto supera todas las líneas base, incluidas las modernas.

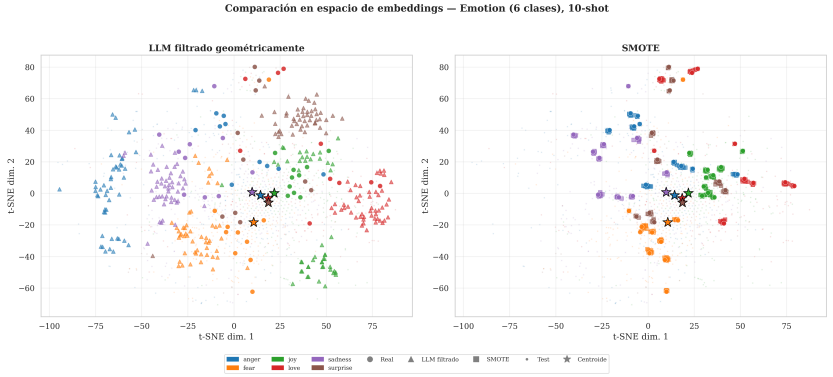
El mapa en acción: antes y después del control de calidad



Noticias, 4 temas, 10 ejemplos reales por clase (círculos). **Izq.:** todo lo que escribió la IA (triángulos). **Der.:** solo lo que pasó el filtro.

El filtro descarta lo que cae entre medio de dos clases o en la periferia; conserva lo que refuerza al grupo.

En el mismo mapa: SMOTE vs. nuestro filtro

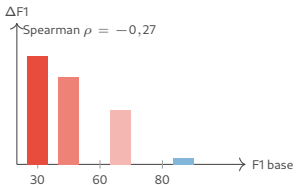


Emociones, 6 clases, 10 ejemplos por clase: triángulos = textos de IA filtrados, cuadrados = puntos de SMOTE.

SMOTE se apila sobre lo conocido; la IA filtrada agranda cada grupo sin invadir a los vecinos.

Resumen por dificultad: la mejora escala con el déficit

Dificultad (F1 base)	N clases	Δ medio
Difícil (<30 %)	4	**+10.44**
Media (30–60 %)	4	+8.45
Fácil (60–80 %)	4	+5.40
Muy fácil (>80 %)	1	+0.55



Leído en el mapa:

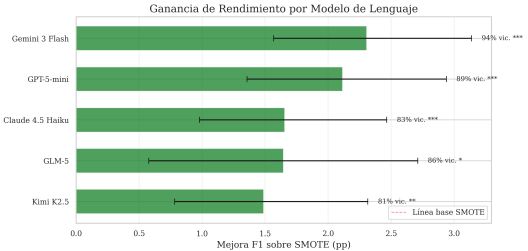
- Las clases difíciles viven en zonas de frontera, donde se mezclan con otras clases.
- Los ejemplos filtrados **pueblan la zona correcta** de cada clase.
- Eso aclara el límite entre clases justo donde el modelo más dudaba.

Implicancia práctica

Cuanto peor parte una clase, más la ayuda el método: la mejora llega sola **adonde más se necesita**.

Mayor déficit inicial ⇒ mayor ganancia: la mejora va a buscar a las clases débiles.

Robustez: ¿importa qué IA escribe, o qué mapa se usa?



5 IAs generadoras probadas

Gemini, GPT-5, Claude, Kimi, GLM

Todas mejoran: entre +1.49 y +2.31 pp

Sin diferencias significativas entre ellas

4 «mapas» (embeddings) distintos

Modelo de mapa	Mejora
mpnet-base-v2	+2.74 pp
bge-large	+2.04 pp
bge-small	+2.00 pp
e5-large	+0.97 pp

Los 4 mapas coinciden en el veredicto el 90.5 % de las veces.

Da lo mismo qué IA escribe y qué mapa se usa: el método funciona igual.

7.
Discusión y aplicaciones

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

¿Cuándo y cómo usarlo? Utilidad práctica

Receta de aplicación:

1. Tienes 10–25 ejemplos por clase.
2. Tienes 4–10 clases.
3. Pides a una IA el triple de ejemplos.
4. Filtras por **cercanía** en el mapa (nivel 1).
5. Pesas por calidad y entrenas un clasificador **lineal**.

Costo real

$\sim$ 10–50 llamadas LLM por clase.

A precios actuales: **centavos por dataset**.

Posibles casos de aplicación:

- Detección de discurso de odio [hate_speech: +13 pp en clase rara].
- Clasificación de emociones en texto [emotion: +4.55 pp; +16.9 pp en anger].
- Categorización de noticias por tema [20newsgroups, ag_news: +5 a +15 pp].
- Clasificación de tipo de preguntas [trec6: +10.12 pp].
- Detección de spam SMS [sms_spam: $\sim +3$ pp].

Quién no debería usarlo:

- Más de 50 ejemplos por clase: lo clásico basta.
- $\geq$ 77 clases: el mapa se fragmenta.
- Bosques aleatorios como único clasificador (XGBoost sí sirve).

Limitaciones, trabajo futuro y cierre

Limitaciones honestas

- Evaluación solo en **inglés**.
- Parte de los textos de IA se reutilizó entre repeticiones; verificado aparte: la mejora **persiste** sin ese atajo.
- Pierde efectividad con ≥ 77 clases.
- Las IAs comerciales se niegan a escribir contenido sensible (p. ej. discurso de odio), justo donde más falta hace.

Trabajo futuro

- Probar en **español** y otros idiomas.
- IAs locales (Llama, Mistral), sin restricciones de contenido.
- Que la exigencia de la curva de pesos se ajuste **sola por clase**.
- Otras tareas: responder preguntas, resumir…

Gracias.

Preguntas, comentarios, contraejemplos: bienvenidos.

Referencias i

Sobremuestreo geométrico clásico

- Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. J. Artif. Intell. Res., 16, 321–357.
- He, H., Bai, Y., Garcia, E. A. & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. IEEE IJCNN, 1322–1328.

Aumentación textual ligera

- Wei, J. & Zou, K. (2019). EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. EMNLP-IJCNLP, 6382–6388.
- Sennrich, R., Haddow, B. & Birch, A. (2016). Improving Neural Machine Translation Models with Monolingual Data. ACL.
- Edunov, S., Ott, M., Auli, M. & Grangier, D. (2018). Understanding Back-Translation at Scale. EMNLP, 489–500.
- Kobayashi, S. (2018). Contextual Augmentation: Data Augmentation by Words with Paradigmatic Relations. NAACL-HLT, 452–457.
- Wu, X., Lv, S., Zang, L., Han, J. & Hu, S. (2019). Conditional BERT Contextual Augmentation. ICCS, 84–95.

Aumentación en espacios de embeddings

- Zhang, H., Cisse, M., Dauphin, Y. N. & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. ICLR.
- Chen, J., Yang, Z. & Yang, D. (2020). MixText: Linguistically-Informed Interpolation of Hidden Space for Semi-Supervised Text Classification. ACL, 2147–2157.
- Bayer, M., Kaufhold, M. A. & Reuter, C. (2023). A Survey on Data Augmentation for Text Classification. ACM Computing Surveys, 55(7), 1–39.

Referencias ii

Generación con LLMs

- Brown, T. et al. (2020). Language Models Are Few-Shot Learners. NeurIPS, 33, 1877–1901.
- Whitehouse, C., Choudhury, M. & Aji, A. F. (2023). LLM-powered Data Augmentation for Cross-lingual Commonsense Reasoning. ACL, 1234–1245.
- Suhaeni, C. & Yong, H.-S. (2023). Mitigating Class Imbalance in Sentiment Analysis through GPT-3-Generated Synthetic Sentences. Appl. Sci., 13, 9766. (+12,7 % acc. en minoritarias.)
- Gopali, S., Abri, F., Namin, A. S. & Jones, K. S. (2024). The Applicability of LLMs in Generating Textual Samples for Analysis of Imbalanced Datasets. IEEE Access, 12, 136451–136465.
- Cegin, J., Simko, J. & Brusilovsky, P. (2025). LLMs vs Established Text Augmentation Techniques for Classification: When do the Benefits Outweigh the Costs? NAACL, 10543–10570.
- Springer, J. M., Devoto, C., Tosheva, M. I. & Ramesh, P. (2025). On the Influence of Synthetic Data for Few-Shot Text Classification. Findings ACL, 8249–8268.

Fine-tuning de LLMs para generación sintética

- Cloutier, N. A. & Japkowicz, N. (2023). Fine-tuned generative LLM oversampling can improve performance over traditional techniques on multiclass imbalanced text classification. IEEE Big Data, 5181–5186.
- Nguyen, D. et al. (2024). Large Language Models for Imbalanced Classification: Diversity makes the difference (ImbLLM). arXiv:2510.09783.
- Sundararajan, N. T. & Kumarapathirage, A. (2023). SentiGEN: A Sentiment-Aware Generative Model for Text Data Augmentation. Findings EMNLP, 9138–9149.

Referencias iii

Filtrado, anomalías y selección de datos

- Breunig, M. M., Kriegel, H.-P., Ng, R. T. & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. ACM SIGMOD, 93–104.
- Robinson, M., Dey, S. & Chiang, T. (2025). Token Embeddings Violate the Manifold Hypothesis. NeurIPS.
- Tessari, F. & Hogan, N. (2024). Surpassing Cosine Similarity for Multidimensional Comparisons: Dimension Insensitive Euclidean Metric. arXiv:2407.08623.
- Du, Y. et al. (2025). Disentangling the Roles of Representation and Selection in Data Pruning. ACL, 16791–16809.
- Yu, T., Zhuang, Y., Zhang, Z., Socher, R. & Xiong, C. (2025). R.I.P.: Better Models by Survival of the Fittest Data. ICML.
- Liu, Z., Tang, Y., Wang, H. & Li, M. (2025). Residual Alignment Model: A Framework for Importance Sampling in Data Augmentation. NeurIPS.
- Tokdar, S. T. & Kass, R. E. (2010). Importance Sampling: A Review. WIREs Comput. Stat., 2(1), 54–60.

NER y extensiones

- Zhou, R. et al. (2022). MELM: Data Augmentation with Masked Entity Language Modeling for Low-Resource NER. ACL, 2251–2262.

Manifold hypothesis y embeddings

- Bengio, Y., Courville, A. & Vincent, P. (2013). Representation Learning: A Review and New Perspectives. IEEE TPAMI, 35(8), 1798–1828.
- Fefferman, C., Mitter, S. & Narayanan, H. (2016). Testing the Manifold Hypothesis. J. Amer. Math. Soc., 29(4), 983–1049.
- Muennighoff, N., Tazi, N., Magne, L. & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. EACL, 2014–2037.

Backup B0a: ¿Por qué este protocolo estadístico?

Cada herramienta responde una pregunta **distinta** sobre el efecto:

Herramienta	Pregunta	Justificación
t-pareada	¿La diferencia es real?	Configs emparejadas por dataset $\times$ clasificador $\times$ semilla: elimina varianza entre configs.
Bonferroni	¿Y con muchos tests?	7 métodos vs. SMOTE $\Rightarrow \alpha = 0.05/7$. Conservador, blindado contra falsos positivos.
Bootstrap CI 95 %	¿Qué rango es plausible?	No asume normalidad. 10 000 remuestreos: percentiles 2.5 y 97.5 estables al 3er decimal.
d de Cohen	¿Qué tan grande es?	Separa significancia **estadística** de **práctica**: con n=525, p puede ser bajo aunque Δ sea trivial.
Win-rate	¿Qué tan consistente?	Media $+2.25$pp con 50 % win = volátil. Con 82 % win = robusto.

Los **4 apuntan** en la misma dirección $\Rightarrow$ conclusión difícil de atacar.

Backup B0b: d de Cohen — interpretación

Fórmula (pareada):

$$d = \frac{\overline{\Delta F1}}{SD(\Delta F1)}$$

Convenciones (Cohen 1988):

d	Magnitud	Interpretación
0.2	pequeño	apenas distinguible
0.5	medio	visible a ojo
0.8	grande	claramente diferente
1.0+	muy grande	casi sin solapamiento

Nuestros valores

Configuración	d	Magnitud
Global (soft-weighted)	0.74	medio-grande
SVC (lineal)	1.00	muy grande
Ridge	1.03	muy grande
MLP	0.73	medio-grande
LogReg	0.82	grande
emotion 6-clases	1.24	muy grande
20news 10-shot	0.95	muy grande

Con $d = 0{,}74$ y $p < 0{,}0001$: la diferencia es **real** y **práctica**.

Backup B0c: Robustez no-paramétrica (Wilcoxon)

La t pareada supone normalidad (aprox.) de las diferencias. El **Wilcoxon de rangos con signo** no supone nada sobre la distribución: si ambos coinciden, la conclusión **no depende del supuesto**.

Resultado principal (N=525: 21 datasets × 5 clf × 5 semillas)

vs. SMOTE	Δpp	win	p (t)	p (Wilcoxon)
soft_weighted	+2.25	82 %	1.5×10^{-49}	4.3×10^{-54}
binary_filter	+2.11	78 %	5.6×10^{-44}	6.9×10^{-46}

Por n-shot (Wilcoxon, soft_weighted)

n-shot	Δpp	p (Wilcoxon)	d
10	+4.52	6.3×10^{-25}	1.16
25	+1.88	2.0×10^{-26}	1.16
50	+0.34	1.1×10^{-5}	0.22

Implicancia

Wilcoxon coincide con la t y da p **aún menor**. El win-rate (82 %) es la prueba de signos asociada. En 50-shot sigue significativo pero con efecto **pequeño** (d=0,22) — coherente con que el método es para **muy bajo recurso**.

Backup B2: Asimetría de semillas y caching

Observación: ≈80 % de las combinaciones LLM × clasificador determinista tienen **varianza cero** entre semillas (la generación LLM está cacheada ⇒ los lineales dan F1 idéntico).

Validación 1 — subset estocástico (exp. principal, MLP+RF, varianza genuina):

- ΔF1 $= +1.79$ pp, p $= 0.0015$, d $= 0.52$.
- Wilcoxon p $= 7 \times 10^{-13}$. Significancia se mantiene.

Validación 2 — re-runs RF/XGB (`tuning_v2`, clasificadores estocásticos corridos de cero):

re-run (var. real)	Δpp vs SMOTE	p (Wilcoxon)
Random Forest	$+1.50$	0.012
XGBoost	$+9.10$	$< 10^{-9}$

Solo 22 % de varianza cero (vs ≈80 % cacheado): la varianza cross-seed es **real**.

Implicancia

Los CIs del exp. principal subestiman la varianza, pero las pruebas t pareadas **siguen válidas** (mismo split por par). Y donde la varianza es **genuina** (re-runs), el efecto no solo sobrevive: **crece** (XGBoost $+9.10$).

Si ese subset usa un árbol que sí pondera (prom. MLP $+2.16$ / RF $+1.50$ / XGB $+9.10$): de $+1.79$ a $\approx$ **$+4.3$ pp**.

Backup B0d$_3$: ¿Filtro binario o peso continuo? (Random Forest)

Las formas de usar las muestras del LLM, en Random Forest (ΔF1 macro vs SMOTE):

Modo	Δ vs SMOTE	
`no_augmentation` (sin aumentar)	−0,70pp	↓ peor que SMOTE
`llm_unfiltered` (sin filtro)	+1,39pp	
`binary_filter` (keep/reject)	**+1,89** pp	↑ el mejor
`soft_weighted` (peso continuo)	+1,50pp	↓ no mejora sobre binario

Re-ejecución `tuning_v2` (clasificador fijo, varianza real, con control `llm_unfiltered`).

En RF el aporte está en **seleccionar** (binario +1,89); **ponderar no agrega** (paso −0,39pp, n.s.): el bagging usa árboles profundos que memorizan sin importar el `sample_weight`. Opuesto a XGBoost (B0d$_4$): bagging → selección, boosting → ponderación.

Backup B0d$_4$: ¿Filtro binario o peso continuo? (XGBoost)

Las formas de usar las muestras del LLM, en XGBoost (ΔF1 macro vs SMOTE):

Modo	Δ vs SMOTE	
`no_augmentation` (sin aumentar)	−0,98pp	↓ peor que SMOTE
`llm_unfiltered` (sin filtro)	+7,96pp	
`binary_filter` (keep/reject)	+7,52pp	↓ peor que sin filtro
`soft_weighted` (peso continuo)	**+9,10** pp	↑ el mejor

Seleccionar/descartar (binario) **no aporta**; **ponderar por el score sí**. Usá el filtro geométrico como **peso continuo**, no como keep/reject.

Backup B0d$_5$: Desglose — ¿seleccionar o ponderar?

Del +2.25pp de `soft_weighted` vs SMOTE: ¿cuánto es **seleccionar** (filtro binario) y cuánto el **paso** de **ponderar** (peso continuo)?

Por familia de clasificador (ΔF1 vs SMOTE):

Familia	Binario (selección)	Suave (completo)	Paso suave (suave−bin.)
No-árbol [a]	+2.27	+2.46	+0.19
Bagging (RF) [a]	+1.51	+1.42	−0.09
Boosting (XGB) [b]	+7.52	+9.10	**+1.59*****

[a] exp. principal (No-árbol = LR/SVC/Ridge/MLP; y RF). [b] `tuning_v2`, clf. fijo. *** d=0,92.

Global — los 5 clf (N=525):

Modo	Δpp	win	d
`binary_filter` (selección)	+2.11	78 %	0.69
`soft_weighted` (completo)	+2.25	82 %	0.74
paso de ponderar	+0.13	54 %	n.s.

Ponderación suave a profundidad

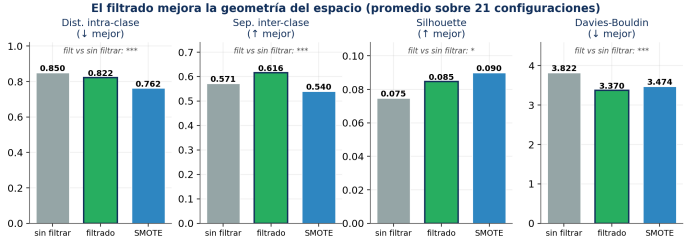
El **paso** de ponderar rinde distinto según el clasificador:

- **Lineal/MLP:** la selección ya quitó los outliers del margen ⇒ ponderar agrega poco.
- **Boosting:** cada árbol corrige el error **ponderado** del previo ⇒ el peso sí se explota (en XGB el paso binario es nulo, −0.44 n.s.).
- **Bagging:** árboles profundos memorizan sin importar el peso ⇒ ningún paso ayuda.

Por n-shot el paso ayuda más en bajo recurso: 10 → +0.36 (66 %), 25 → +0.15, 50 → −0.11.

La slide 20 muestra que el **método completo** gana en todo clasificador; este desglose muestra **por qué**: los lineales explotan la **selección**, el boosting la **ponderación**. La misma señal geométrica entra por canales distintos ⇒ H1b (peso suave) es condicional al clasificador.

Backup B6: ¿El filtro mejora la geometría? (validación intrínseca)



Filtrado mejora vs. sin filtrar

- Dist. intra-clase $0.850 \rightarrow 0.822$ [$p < 0{,}001$]: clústeres más compactos.
- Sep. inter-clase $0.571 \rightarrow 0.616$ [$p < 0{,}001$]: clases más separadas.
- Silhouette $0.075 \rightarrow 0.085$ [$p = 0{,}029$]; Davies-B. $3.822 \rightarrow 3.370$.

El matiz de SMOTE

SMOTE es **más compacto** (intra 0.762, silhouette 0.090) porque **interpola** dentro del convex hull. Pero tiene la **peor separación** inter-clase (0.540) y peor Davies-Bouldin (3.474) que el filtrado.

El filtro **cierra el lazo causal**: mejora la geometría intrínseca, no solo el F1. La compacidad artificial de SMOTE **no se traduce** en mejor clasificación; la **diversidad + separación** del LLM filtrado sí.

Backup B0f: Comparación completa de filtros geométricos

Filtro	ΔF1	Win	Selección	LLM	Notas
cascade L1 + top-N	+3.66	84.5 %	ranking top-N	3×	**Ganador**
cascade L2 (top-N)	+3.60	83.9 %	ranking top-N	3×	Similar a L1
cascade L4 (top-N)	+3.60	83.3 %	ranking top-N	3×	Redundante
cascade L1 (binary)	+1.94	91.7 %	ranking top-N	3×	Sin pesos suaves
cascade L3 (binary)	+1.95	91.7 %	ranking top-N	3×	+ KNN, no ayuda
LOF (k=5, θ=−0.5)	+2.43	90.0 %	umbral (relajado)	6.2×	Segundo mejor
LOF (k=20, θ=0)	+0.05	67.5 %	umbral (estricto)	8×	Default sklearn falla
none (control)	+0.72	62.9 %	sin filtrar	3×	LLM directo
embedding_guided	−1.29	55.0 %	cobertura greedy	–	Coverage-only falla
combined (LOF ∧ sim)	−3.28	52.5 %	AND, ∼23 % pasa	32×	Demasiado restrictivo

Selección: ranking top-N toma los N mejores del pool 3× generado (filtrado por ranking, no por umbral). Umbral: rechaza candidatos bajo el score; el % indica cuántos pasan. "LLM" = candidatos por muestra requerida.

Tres principios confirmados: (1) Ranking top-N $>$ umbral duro. (2) Distancia simple $\geq$ múltiples criterios. (3) Operador **AND** destruye la señal.

Backup B0e: Elección de hiperparámetros

LOF: `n_neighbors` y `threshold`

Param	k = 5	k = 10	k = 20
ΔF1 vs. SMOTE	+2,43	+0,16	+0,05
Win-rate	90.0 %	70.0 %	67.5 %

Default sklearn k=20 falla en 10-shot (no hay 20 vecinos por clase). k=5 es el sweet spot.

LOF: `threshold`

Threshold	ΔF1	Win-rate
−0,50	+1,29	80.0 %
−0,30	+0,06	68.3 %
0,00 (default sklearn)	+0,88	75.6 %

θ=0 es la frontera matemática inlier/outlier (*`decision_function`*).

Cascade: `k_neighbors` para KNN purity

- Fijado k=10 (compromiso 10/25/50-shot).
- L1 (sin KNN) no es sensible a este parámetro.

Combined: `sim_threshold`

θ_{sim}	ΔF1	Win-rate
0,3	−0,68	55.0 %
0,5	−5,71	41.7 %
0,7	−1,10	71.7 %

Cualquier $\theta_{sim} > 0$ combinado con AND $\Rightarrow$ catastrófico.

Top-N: peso mínimo y selectividad

- Normalización: `minmax` (vs. rank, z-score).
- Selectividad τ=0,5 (peaked, $w \propto s^{1/\tau}$).
- Peso mínimo $w_{mín}$: **poco impacto** ($\pm$0,05pp).

Hiperparámetros del cascade son **poco sensibles**; los de LOF y combined son **críticos**.

Backup B0g: Hiperparámetros LLM y ponderación suave

LLM (Gemini 3 Flash)

Param	Valor	Justificación
`temperature`	0,8	Diversidad / coherencia
`top_p`, `top_k`	default	SDK Gemini
`max_tokens`	4000	$\sim$ 50 muestras/llamada
back-trans. T	0,3	EN $\leftrightarrow$ ES conservador

Ablation `temperature` (cascade L1):

T	sms_spam	20news
0,5	$-0{,}07$	$+5{,}04$
0,7	$+1{,}44$	$+4{,}16$
0,8	$-1{,}06$	$+6{,}04$
0,9	$-0{,}18$	$+6{,}25$
1,0	$+4{,}95$	$+7{,}44$

No monótono. T=0,8 por convención.

Ponderación suave (top-N)

Param	Valor	Comentario
τ ponder.	0,5	$w \propto s^{1/\tau}$, peaked
normalización	minmax	frente a rank, sigmoid
$w_{mín}$	0,0	sensibilidad $< 0{,}2$pp
candidatos	$3\times$ target	margen top-N

Sweep Phase 2 (468 configs):

Param	ΔF1	Win
$\tau = 0{,}5$	$+4{,}10$	88.3 %
$\tau = 1{,}0$	$+3{,}67$	84.0 %
$\tau = 2{,}0$	$+3{,}41$	83.3 %
norm. minmax	$+3{,}98$	89.6 %
norm. rank	$+3{,}56$	83.3 %

τ=0,5 **peaked** amplifica diferencias entre muestras buenas y mediocres $\Rightarrow$ máxima ganancia.

Backup B0i: Hiperparámetros de los clasificadores

Experimento principal — 5 clasificadores

Defaults sklearn / PyTorch, sin tuning por dataset:

Modelo	Hiperparámetros
Logistic Reg.	L2, C=1.0, `lbfgs`, `max_iter=1000`
SVC	`kernel=linear`, C=1.0
Ridge	α=1.0
Random Forest	`n_estimators=100`, `max_depth=None`, `class_weight=None`, `max_features=sqrt`
MLP *	1 capa oculta = 128, ReLU, `dropout=0.3`, Adam lr= 10^{-3}, `weight_decay`= 10^{-4}, cosine, 300 épocas, batch 16

* implementación propia (PyTorch); pérdida CE con `sample_weight`. Todos: `random_state` por semilla; muestras reales ⇒ w=1.

Análisis de árboles (RF / XGBoost)

Fijos (referencia):

- RF = el del principal (`n_estimators=100`).
- XGB: `n_est=100`, `depth=6`, lr= 0.1, `subsample=colsample=1.0`, `hist`.

Selección por CV — `StratifiedKFold(3)` sobre **train** (sin fuga al test):

	Grilla (# combos)
RF cv	`n_est=300`; `depth`∈ {6, 10, 15}; `min_leaf`∈ {1, 3}; `cw`∈ {None,bal.} (12)
XGB cv	`depth`∈ {3, 5}; `min_child`∈ {1, 3}; `subsample`∈ {0,8, 1}; `colsample`= 0,8 (8)

Defaults a propósito en el principal: las diferencias miden el **método de aumentación**, no el ajuste del clasificador. El tuning de árboles usa CV **solo en train** (el test se toca una vez) ⇒ neutraliza el ataque "ajustaste XGBoost en el test". LLM y peso suave: B0g; umbrales de filtros: B0e.

Backup B0h: Embeddings, clasificadores y protocolo

Clasificadores (sklearn defaults)

Modelo	Hiperparámetros
LogReg	`max_iter=1000`
SVC	`kernel="linear"`
RandomForest	`n_estimators=100`
Ridge	$\alpha = 1{,}0$
MLP (custom)	128 hidden, 300 ep., lr 10^{-3}

Sin tuning por dataset $\Rightarrow$ las diferencias son del **método de aumentación**.

Embedding model (504 runs)

Modelo	Dim	ΔF1	Win
mpnet-base	768	+2,74	92.1 %
bge-large	1024	+2,04	69.8 %
bge-small	384	+2,00	76.2 %
e5-large	1024	+0,97	68.3 %

Más grande $\neq$ mejor: bge-small (384d) supera a e5-large (1024d).

Protocolo

Seeds $\{42, 123, 456, 789, 1011\}$ · N-shot $\{10, 25, 50\}$ · LLM % $\{5, 25, 50, 100\}$ · macro F1 · 50 sintét./clase · cache MD5.

Hiperparámetros del **filtro** son robustos ($\pm 0{,}5$pp). Los del **embedding** y **clasificador** dominan la varianza.

Backup B5: Comparación con baselines modernos

1.890 configuraciones, 10 métodos, 3 clasificadores:

Método	ΔF1 vs. SMOTE	p Bonf.	Win-rate
Soft-weighted (ours)	+2.61	< 0.0001	88.9 %
Binary-filter (ours)	+2.39	< 0.0001	79.4 %
No augmentation	+0.64	< 0.05	56.0 %
Embedding mixup	−0.01	n.s.	51.0 %
Contextual BERT	−0.26	n.s.	48.0 %
T5 paraphrase	−0.35	n.s.	47.0 %
Random oversample	−0.32	n.s.	47.0 %
EDA	−0.49	< 0.05	41.0 %
Back-translation	−0.50	< 0.05	40.0 %

Todas las técnicas modernas **pierden** contra SMOTE; solo el filtrado geométrico gana.

Backup B11: Las clases que más ganan — las que partían perdiendo

Dataset	Clase	SMOTE	Ours	Δ
Clases con F1 base bajo				
emotion	**anger**	27.8	44.6	**+16.9**
20news	**alt.atheism**	50.9	65.9	**+15.0**
emotion	joy	29.4	42.6	+13.3
hate_speech	**offensive**	73.3	86.3	+13.0
emotion	fear	48.8	59.0	+10.2
Clases con F1 base alto				
20news	comp.graphics	90.8	91.3	+0.5
emotion	sadness	41.3	42.3	+1.0

10-shot; 3 clasificadores lineales.

Patrón: las clases difíciles inicialmente son las que más ganan con filtrado geométrico.

- "anger" arranca en **27.8 %** (poco discriminable).
- Termina en **44.6 %**: +16,9 pp.
- "comp.graphics" parte ya en 90.8 %: poco margen, ganancia +0,5 pp.

Donde el clasificador **ya rinde bien**, el filtrado aporta poco.
Donde **tropieza**, el filtrado densifica la región correcta y refuerza la frontera.

El método actúa justo donde el modelo más lo necesita.

Backup B10: ¿Por qué estos datasets de clasificación?

Experimento principal — 7 datasets

Buscan **cobertura en tres ejes**:

- **Granularidad:** 2 → 20 clases.
- **Dominio:** spam, odio, noticias, emoción, ontología.
- **Estilo:** corto/largo, formal/informal, social/curado.

- `sms_spam` (2): NLP canónico, frase corta.
- `hate_speech_davidson` (3): dominio sensible.
- `20newsgroups`, `ag_news` (4): noticias.
- `emotion` (6): el ejemplo recurrente.
- `dbpedia14` (14): ontología Wikipedia.
- `20newsgroups_20class` (20): máxima granularidad.

Escalabilidad — 3 datasets adicionales

Responden la pregunta:

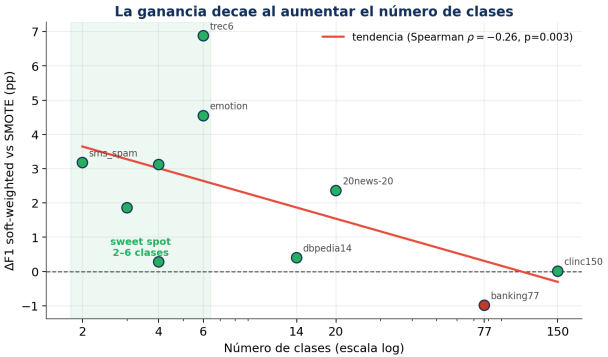
¿el filtrado sigue funcionando con muchas clases?

- `trec6` (6): preguntas TREC, mismo régimen de clases que el principal pero distinto dominio. Sirve de **control**.
- `banking77` (77): intenciones bancarias, granularidad fina con clases semánticamente cercanas.
- `clinc150` (150): intenciones de diálogo de asistentes virtuales. **Caso extremo**.

Permiten medir **degradación del filtrado** al aumentar el número de clases.

Diez datasets, dos preguntas: ¿qué tan bueno es el filtrado? ¿hasta dónde escala?

Backup B9: Escalabilidad — ¿por qué decae con muchas clases?



La ganancia cae con más clases.

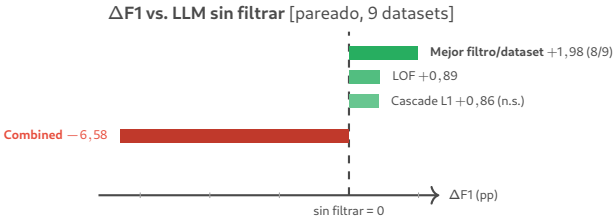
- Sweet spot **2–6 clases**: $+3$ a $+7$pp.
- dbpedia14 (14): $+0.40$; banking77 (77): -0.99; clinc150 (150): $+0.01$.
- Spearman $\rho = -0.260$ [p $=$ 0,003, n $=$ 132].

Mecanismo

Con muchas clases el espacio se **fragmenta**: todo cae "lejos" del ancla y la distancia pierde poder discriminativo. Además, clases semánticamente cercanas (banking77) se solapan.

Límite por diseño: el método es para **bajo-recurso con pocas clases** (2–6), no para taxonomías de 77+ clases (ahí convienen enfoques jerárquicos).

Backup B13: NER — ¿cuánto aporta el filtro sobre el LLM sin filtrar?



El filtro fijo agrega poco (no-sig.); el AND (combined) **sobre-filtra**.

El grueso lo aporta el dato LLM

F1 macro medio (9 datasets):

sin aumentar **23.3** $\xrightarrow{+8,4}$ LLM s/filtrar **31.7** $\xrightarrow{+0,9}$ filtrado **32.6**

Filtrado vs sin filtrar por n-shot:

n-shot	Δ (cascade L1)
10	+1,18
25	+1,89
50	−0,48

Igual que en clasificación: el filtro ayuda en bajo-recurso.

El titular de la slide principal ($\sim +9{,}6$ vs sin aumentar) combina **dato LLM** ($+8{,}4$) y **filtro**. El aporte marginal del filtro sobre el LLM sin filtrar es **modesto** ($+0{,}9$ fijo, n.s. $\rightarrow +1{,}98$ mejor-por-dataset, 8/9), pero **positivo y en otra tarea sin modificar el filtro**: la prueba más fuerte de generalización.

Backup B3: Closed-loop vs. pasada única

Hipótesis exploratoria: ¿mejora si re-prompteamos al LLM con feedback de los rechazos?

Estrategia	ΔF1 vs. SMOTE	Win-rate
Closed-loop (5 iteraciones)	+1.12	52 %
Pasada única ⋆	+2.08	80 %

¿Por qué falla closed-loop?

- Tasas de aceptación **decrecen** con iteraciones (-12.3 pp).
- Fallo dominante: `DISTANCE_OUTLIER` (77 %).
- Cuello de botella: **el LLM**, no el prompt.

Pasada única + filtro simple > ingeniería iterativa de prompt.

Backup B4: Filtrado geométrico vs. teacher

Comparación con un filtrado supervisado (teacher classifier):

Filtro	ΔF1 vs. SMOTE	p	Costo extra
Geométrico (binary)	$+2.78$	< 0.001	ninguno
Teacher (binary)	$+2.43$	< 0.001	entrenar clf adicional

El filtro geométrico es **más general** (no se especializa en separabilidad lineal) y **más eficiente** (no requiere entrenamiento extra).

Backup B12: ¿Estrategias más “inteligentes” ganan? No.

Probamos dos formas de aplicar el filtro **más sofisticadas** que “a todas las clases por igual”. Ninguna supera a la versión simple y uniforme.

1. Por etapas (curriculum)

Incorporar las candidatas del LLM **de a poco**, de las más fáciles a las más difíciles, en vez de todas juntas.

Estrategia	ΔF1 vs SMOTE
Todas a la vez (simple)	+2.61
Por etapas (la mejor)	+2.74

Ganancia de las etapas: solo +0.13pp, a costa de más complejidad.

2. Mezcla LLM/SMOTE adaptada por clase

Decidir **para cada clase** cuánto usar del LLM y cuánto de SMOTE (según distintas reglas), en vez de la misma receta para todas.

¿Cómo se decide la mezcla?	Δ	Win
Igual para todas (uniforme)	+2.33	91 %
Ajustada por clase (CV)	+0.77	36 %
Ajustada por clase (oráculo)	+0.44	29 %
Ajustada por clase (mediana)	−1.11	52 %

Cuanto más se “especializa” la mezcla por clase, **peor** (incluso el “oráculo” ideal).

Aplicar el filtro **uniforme y simple** gana. Ni el curriculum por etapas, ni la mezcla adaptada por clase, ni los filtros multi-criterio (combined −3.28), ni los niveles extra de la cascada (L2–L4 ≈ L1) **superan** la versión simple. Es el mismo hallazgo central: simple > complejo.

Backup B8: Circularidad del embedding — tres defensas independientes

Crítica anticipada: "Usás *all-mpnet* para filtrar Y como features del clasificador: el filtro favorece artificialmente su propio espacio."

Defensa	Por qué rompe el círculo	Resultado
(1) 4 embeddings distintos	filtrar+clasificar con mpnet, bge-large, bge-small, e5	$+2.74$ a $+0.97$pp; **90.5 % acuerdo**
(2) Teacher independiente	filtrar con un clasificador aparte, no el espacio geométrico	geom. $+2.78 \geq$ teacher $+2.43$
(3) NER, pipeline externo	evaluación con SpaCy + seqeval (independiente del embedding)	cascade L1 $+9.61$pp

Tres caminos **independientes** (distinto espacio, distinto filtro, distinta tarea+evaluación) dan la misma conclusión: la mejora **no** es un artefacto del embedding compartido.

Límite honesto

9.5 % de desacuerdo entre embeddings en 50-shot: posible efecto residual marginal.

Backup B1: SMOTE — validación de implementación

Crítica anticipada del comité:

> "¿Su SMOTE dummy-binary es equivalente al SMOTE estándar binary-mask?"

Validación:

- 27 comparaciones lado a lado.
- Diferencia máxima: 0.0000 pp.
- F1 idénticos en todos los datasets.

Línea base SMOTE **robusta y reproducible**.

Resumen

Implementación	F1 medio
dummy-binary	0.7124
binary-mask std.	0.7124
Diferencia	**0.0000**